

20.06.25

What am I to do this week?

Well, for now, we decided to fully focus on Jonas' encoding, so that after his departure I can dedicate almost two months to my engineering experiments. Now, only perfecting/building on top of Jonas' thing. So, the plan is:

1. **Play around with Jonas' encoding more, run on a powerful CPU, take statistics for the report. Test on my instances. Try many trains. Design really hard environments - train-wise and/or structure-wise - to test the robustness of our encodings. +**
2. **Understand its inner structure better. +**
3. **See where it can be optimized further without major changes (waiting only before the junction). +**
4. **Jonas has already implemented prioritization - see how he worked around it. Also: read more on the problem, consult with GPT, read domain-area articles.**
5. **Read through papers suggested by Kep at the beginning. Estimate what we could implement in our project on top of / beyond prioritization.**
6. **Read best reports by other groups and start drafting ours from the journal(s) we wrote.**
7. **Learn to interact with flat-land to obtain visualizations.**
8. **Understand and basically prototype speed in case it's needed (I see it in the fourth test environment - it would also be good to directly message Kep about whether we need speed). Consider acceleration as an extension based on speed.**

—

1. Understanding Jonas' encoding better:

Let's first update my local version to the latest he produced: git already did it for me. Analysing:

```
dir(n;s;e;w).
```

We create a fact atom that holds all the possible directions. It's not an array, but rather an object that can take only one of the 4 possible values.

```
action_type(move_left;move_right;move_forwards;wait).
```

The same, but for a possible action type.

```
time(0..T) :- global(T).
```

A normal rule that says: if there is a global time atom with a single value T, then there must be a range atom 'time' with the the range of values from 0 to T. This is necessary to make time in the system 'move'.

```
global(Max) :- Max = #max { T : end(_,(_,_),T) }.
```

A normal rule that does the following: take the maximum time that is indicated in the 'end' atoms (ignoring the other variables) and, if such extraction is possible ('end' with the required structure exists), create an atom called 'global' that holds this maximum time value.

```
:- train(ID), not goal(ID,_).
```

A constraint that says: there must be no answer set where there is a train with an ID, but there is no 'goal' atom holding this id (disregarding its second attribute). This 'goal' atom will be created later in the code - for asp the order of lines is not important.

```
at(ID,X,Y,T,Dir) :- move(ID,X,Y,T,Dir).
```

A normal rule that says: if there is a 'move' atom that holds the variables:

ID - train's ID

X - vertical(?) coordinate

Y - horizontal(?) coordinate

T - moment of time

Dir - direction of movement

Then we create an 'at' atom with exactly the same variables (this looks redundant, but perhaps the idea is to aggregate information from both movement and waiting into 'at' atoms).

```
at(ID,X,Y,T,Dir) :- start(ID,(X,Y),T,Dir).
```

A normal rule that says: if there is a start atom with certain attributes, create an 'at' atom with exactly the same attributes.

```
goal(ID,T) :- at(ID,X,Y,T,_), end(ID,(X,Y),MaxT), T <= MaxT.
```

A normal rule that says: if there is an 'at' atom (ignoring direction stored in it), 'end' atom and the moment of time T is less than the variable MaxT - create 'goal' atom that holds the ID of the train and the moment of time T, which means simply: the train ID has reached its goal (no matter where it is) at the moment T (and the moment is earlier than MaxT taken from the train's specific 'end' atom).

```
at(ID,X,Y,T+1,Dir) :- action(train(ID),wait,T), at(ID,X,Y,T,Dir), cell((X,Y),_), time(T).
```

A normal rule: if there is a moment of time T, and the cell with coordinates X and Y exists (no matter of which type), and there is an 'at' atom with attributes of the train ID, coordinates X, Y, moment of time T and direction Dir (not matched with anything except the head of the rule) and 'wait' action for this train at this moment, then create an 'at' atom where the X and Y coordinates are the same at the next moment after T. At a high level, this creates the opportunity for waiting.

```
offset(n,-1,0).  
offset(n,-1,-1).  
offset(n,-1,1).
```

These are the possible offsets for the direction 'north'. I need to understand this graphically. For now, it seems confusing that we have three possible offsets for the direction 'north'. To my mind, if a train moves north, the only change must be the one by vertical axis - decreasing by 1 unit in this strange upside down system.

Hypothesis: direction is the *preceding* state of the train, not the one simultaneous with action. In this case, these offsets might be explained by **the location of the next cell the train reaches**. Let's check this hypothesis with some drawing: done. The hypothesis **doesn't hold**: two of the three changes in location aren't possible in one move - only the first works.

Let's consult with an LLM:

You're absolutely right to question this! The offset definitions in the code are indeed a bit confusing because they mix cardinal directions (N/S/E/W) with diagonal movements (NE/NW/SE/SW) despite the system only officially supporting the four primary directions.

Yes, this is confusing. **I'll need to talk to Jonas about this**. I also agree that the actual movements are constrained by the track types. The diagonal movement possibilities could be written by Jonas at the beginning, before he properly understood movement. Let's not dwell on it. There is also a good chance that I will understand his logic from reading the code further.

```
:- action(train(ID),wait,T), at(ID,X,Y,T,Dir),
#count { ID2,DX,DY : offset(Dir,DX,DY), at(ID2,X+DX,Y+DY,T,_), ID>ID2 } = 0.
```

A constraint that says: there must be no answer set where there is 'wait' action by the train with ID at the moment T and there is an 'at' atom for the same train and the same moment with coordinates X and Y and direction Dir such that the count of higher priority trains ($ID > ID2$) occupying the neighbouring cells possible for the current offset (not necessarily the immediately next cell) is 0. Now I'm starting to see the point in offsets: it's not always the 'right next cell', but can also be 'one cell away' - which makes sense practically: for example, the train is before a junction facing north, and the junction can put it into the diagonal cell. The decision of waiting must be taken before the junction, so it makes sense 'to see' two cells ahead.

```
reached(ID,T) :- goal(ID,T2), time(T), train(ID), T > T2.
```

A normal rule that says: if there is a 'goal' atom for the train with ID at the moment T2, there is a moment of time T that is larger than T2 from the 'goal' and there is a train with ID from the 'goal', create an atom named 'reached' to *'put the train that is at the destination out of action'*. So, for example, while the other two trains keep moving, the one that has already reached its indicated goal retains the 'reached' status for the remaining moment of time T:

```
reached(1, 13).
reached(2, 14).
reached(3, 15).
```

This also means that it's not occupying the tracks at its destination station any more. This can also be viewed as the base case for generation: take all trains that aren't 'reached' yet, move them around.

```
1 {action(train(ID),A,T) : action_type(A), A!=wait;
action(train(ID),wait,T) : at(ID,X,Y,T,Dir), offset(Dir,DX,DY),
at(_,X+DX,Y+DY,T,_)} 1 :- train(ID), time(T), not goal(ID,T), not
reached(ID,T).
```

Let's decompose it:

"train(ID), time(T), not goal(ID,T), not reached(ID,T)." - if there is a train with ID and the moment of time T such that the train with ID is not at the goal at the moment of time T and it hasn't reached the goal at the moment T:

```
1 {...} 1 - generate exactly one:
```

`action(train(ID),A,T) : action_type(A), A!=wait;` - action for train with ID at the moment of time T using the fact atom action type but with a constraint that the action is never 'wait'.

`action(train(ID),wait,T) : at(ID,X,Y,T,Dir), offset(Dir,DX,DY),
at(_,X+DX,Y+DY,T,_)` - the 'wait' action is generated separately: if there is an 'at' atom for the train ID where the coordinates are X and Y, the direction is dir and the moment of time is T and there is an offset for this direction with the known axis-offsets DX and DY, and also there is an at atom for ANY other train (disregarding priority) such that its X and Y coordinates are separated from the coordinates of the target train exactly the offset values implied by the target train direction - generate an action atom with 'wait' value for the target (1st) train at the moment T.

IMPLICATIONS: this already means that the train only waits before the junction, with the maximum distance from the junction defined by the constraint.

Communication about what I've see in Jonas' code:

Hey, Jonas, some considerations on your beautiful encoding here:

Given your action generating rule:

```
1 {action(train(ID),A,T) : action_type(A), A!=wait;  
action(train(ID),wait,T) : at(ID,X,Y,T,Dir), offset(Dir,DX,DY),  
at(_,X+DX,Y+DY,T,_)} 1 :- train(ID), time(T), not goal(ID,T), not  
reached(ID,T).
```

1. We *already* have waiting only allowed before the junction and only within the offset distance. So, you don't need to bother trying to enforce the 'only wait once before junction' constraint. You have it implicitly.

2. You have this constraint for priority handling:

```
:- action(train(ID),wait,T), at(ID,X,Y,T,Dir), #count { ID2,DX,DY :  
offset(Dir,DX,DY), at(ID2,X+DX,Y+DY,T,_), ID>ID2 } = 0.
```

I don't if it can positively affect grounding time, but it seems to be packable right into the 'action' generator. I'll try to do this to see what it yields.

edited 19:32 ✓

A potential micro task for myself is to pack the priority constraint right into the generation rule. It feels like it can have a slight positive effect on the ground time. Let's consult with an LLM (DeepSeek):

Yes, integrating the priority check into the choice rule will likely improve grounding time by reducing redundant rule generation. However, if maintainability is a concern, the hybrid approach (using a helper predicate like `action_wait_ok`) is a good compromise.

Done.

Now I need to check if it really improves anything. For this, I need to learn to smoothly run Jonas' encoding with all environments, including mine. Let's do it:

Unlike my encoding, Jonas' one is split into a few files. I've never run asp this way. As far as I remember, it must be run simply as:

```
`clingo 0 file1.lp file2.lp file3l`
```

But rather than copying my environments into another folder, it's easier to recompile Jonas' encoding into one file for testing.

21.06.25

I keep testing Jonas' encoding on my instances. I've already found some failures and it would interesting/important to determine their source:

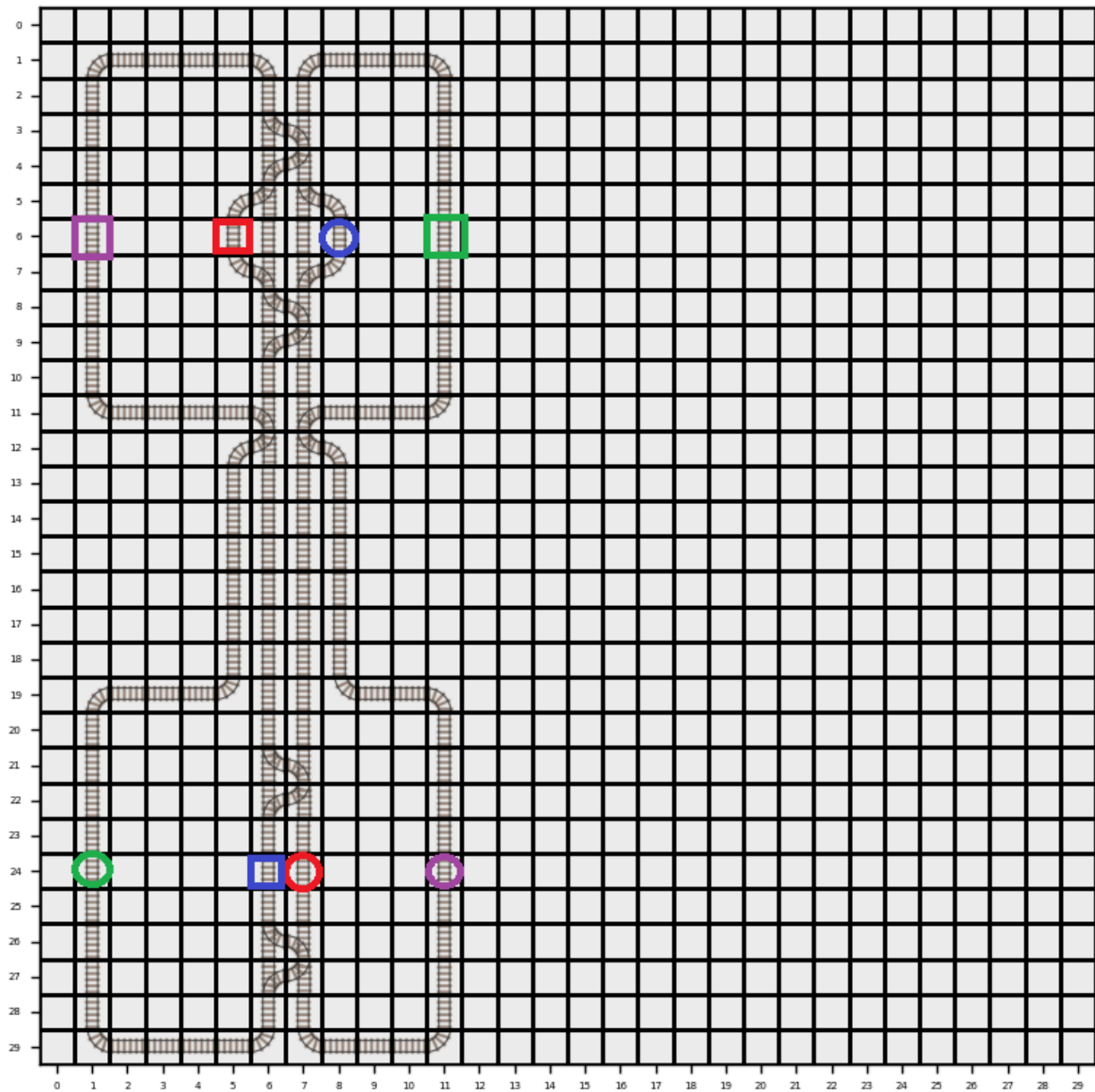
Environment	Result
Straight line	Yes
Г-turn	Yes
Π-turn	Yes
Bagel	No (mine solves)
Many turns	Yes
Ghost train	Yes (doesn't let it run); now running just one train: Yes (no block any more); running the other train: Yes (no block any more).
Switch trap	Yes
Bowel	Yes Let's try to crash them by delaying the start of the second train: Yes

Caduceus	Yes Trying to crash: Yes Shifting times: Yes
Ten by ten	Yes Crashing: yes Shifting: yes There's nowhere to put more trains on this. Also
15x15	This one is broken, but still can be fixed & used for testing & experimenting with extensions
20x20	Also broken and needs fixing. Maybe worth doing for testing and experimenting.
Large Kep's 1	Yes (infinitely faster than mine on this one)
Large Kep's 2	Yes (struggles after the first stable model, trying to find more 'forever'; looking at what mine does: also freezes after the first stable model, but the stable model is found much slower; no more models in many minutes - I interrupt)
Large Kep's 3	Mine: first model's quickly found, then freezes. Jonas': quickly finds 4 optimums, doesn't freeze
Test 40 x 40	Jonas': single stable model found very fast Mine unsatisfiable (debugging later)

NOTE: I've run my latest functional encoding ('motion_5') with '#minimize' imperative, and while fast, it is still slower than Jonas' one on the observed (we can't claim that *on all possible*) environments. Still it's interesting to find where my encoding *even in its current sub-optimized form* surpasses Jonas' one.

Adding more trains to check for robustness:

Kep's Test 1:



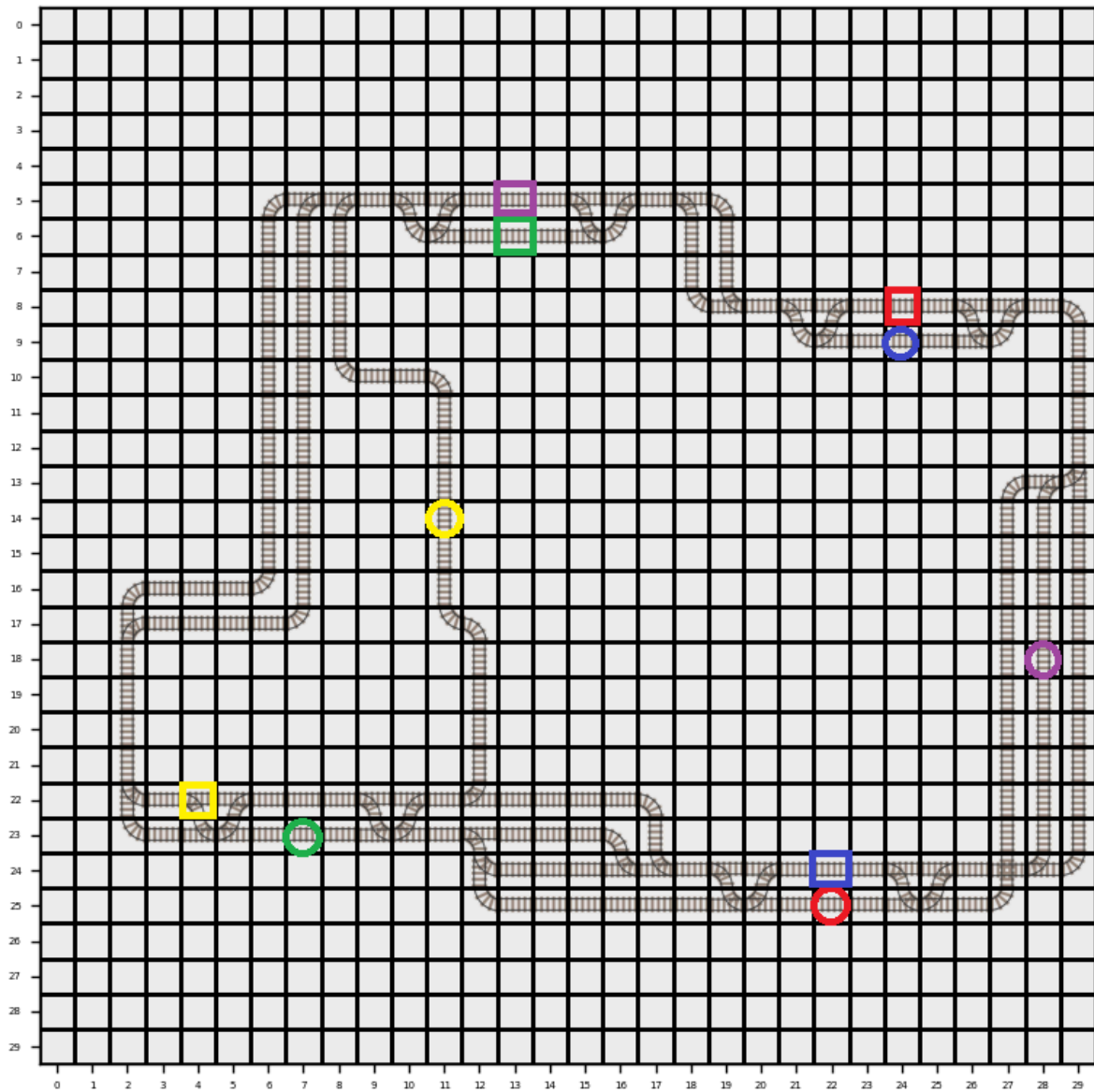
Trains:

```
train(0). start(0,(6,5),0,s). end(0,(24,7),32).  
train(1). start(1,(24,6),2,n). end(1,(6,8),34).  
train(2). start(2,(6,11),2,s). end(2,(24,1),90).  
train(3). start(3,(6,1),2,s). end(3,(24,11),90).
```


Complexity	Result
4 trains	Frozen. Grounding takes longer - surprisingly longer for an addition of just 2 simply positioned trains which don't even contradict the 2 previously existing. Let's run the newly added ones to check if they are 'runnable' at all. Yes, they run - so, there is no bug. Running all 4 again: grounding is long again. UNSATISFIABLE - why? Increased time for the new trains. Solves for 3. Trying 4: fails. Increased time for the second added train. Testing: fails Increased global time: testing: fails.
5 trains	geometrically nonsensical
10 trains	geometrically nonsensical

It proved to be geometrically challenging / practically absurd to put more than 4 trains into this environment. We don't aim for 'raw performance', but rather for solving *realistic environments* - so **there's no point in testing the final encoding on something that doesn't look like a real railway**.

Kep's Test 2:

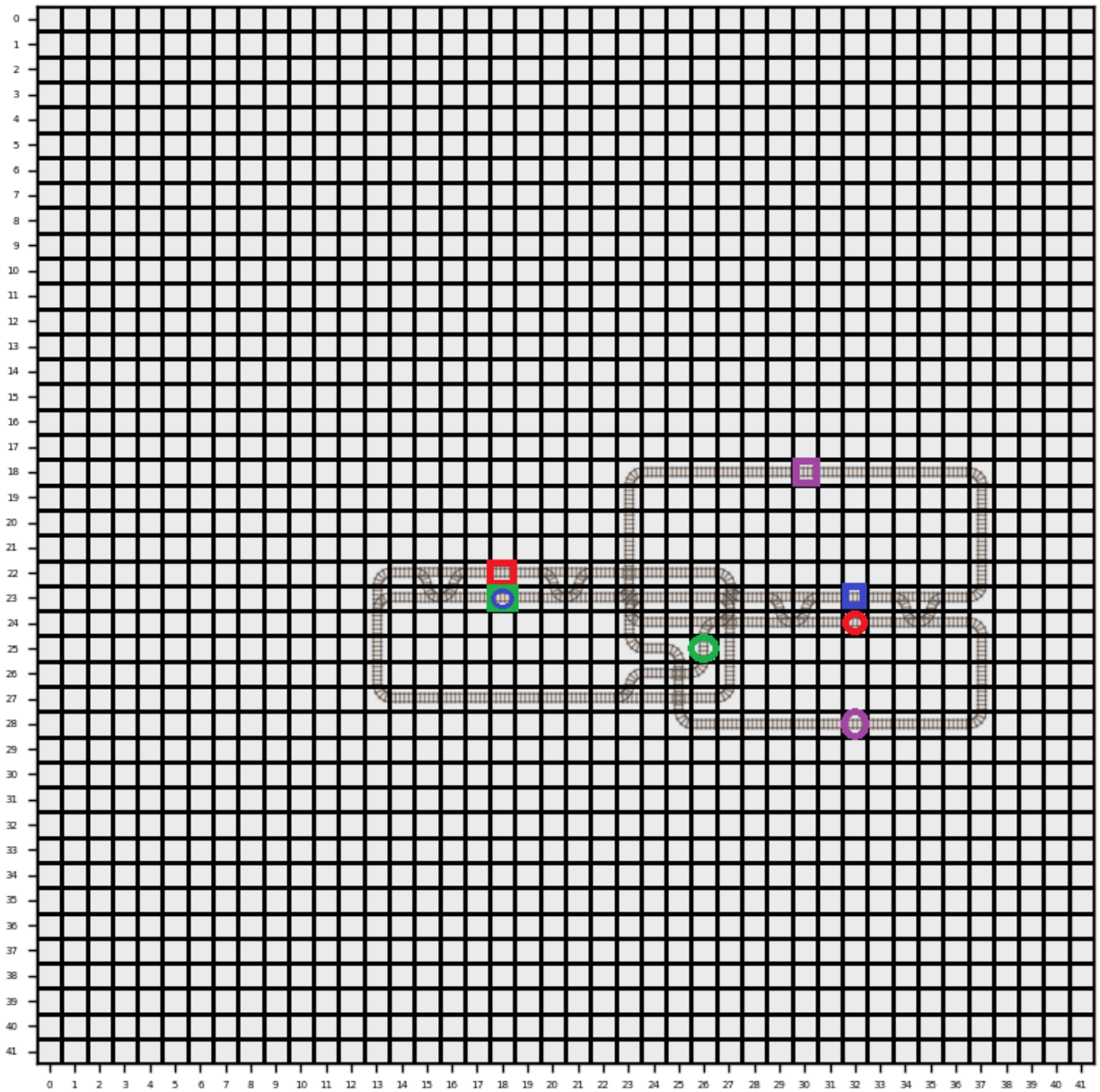


Trains:

```
train(0). start(0, (8,24),2,west). end(0, (25,22),98).  
train(1). start(1, (24,22),8,east). end(1, (9,24),57).  
train(2). start(2, (6,13),0,west). end(2, (23,7),40).  
train(3). start(3, (5,13),0,east). end(3, (18,28),35).  
train(4). start(4, (22,4),0,east). end(4, (14,11),25).
```

Complexity	Result
3 trains	First model found. Then it freezes.
4 trains	Takes some time. A stable model is found. Freezes.
5 trains	Takes a long time to ground. Found a model. Then it freezes. Just for comparison, running it in mine: grounding takes even longer. 'Solving' is displayed. Takes very long time. Solved. Also one answer set, then freezes.

Kep's Test 3:

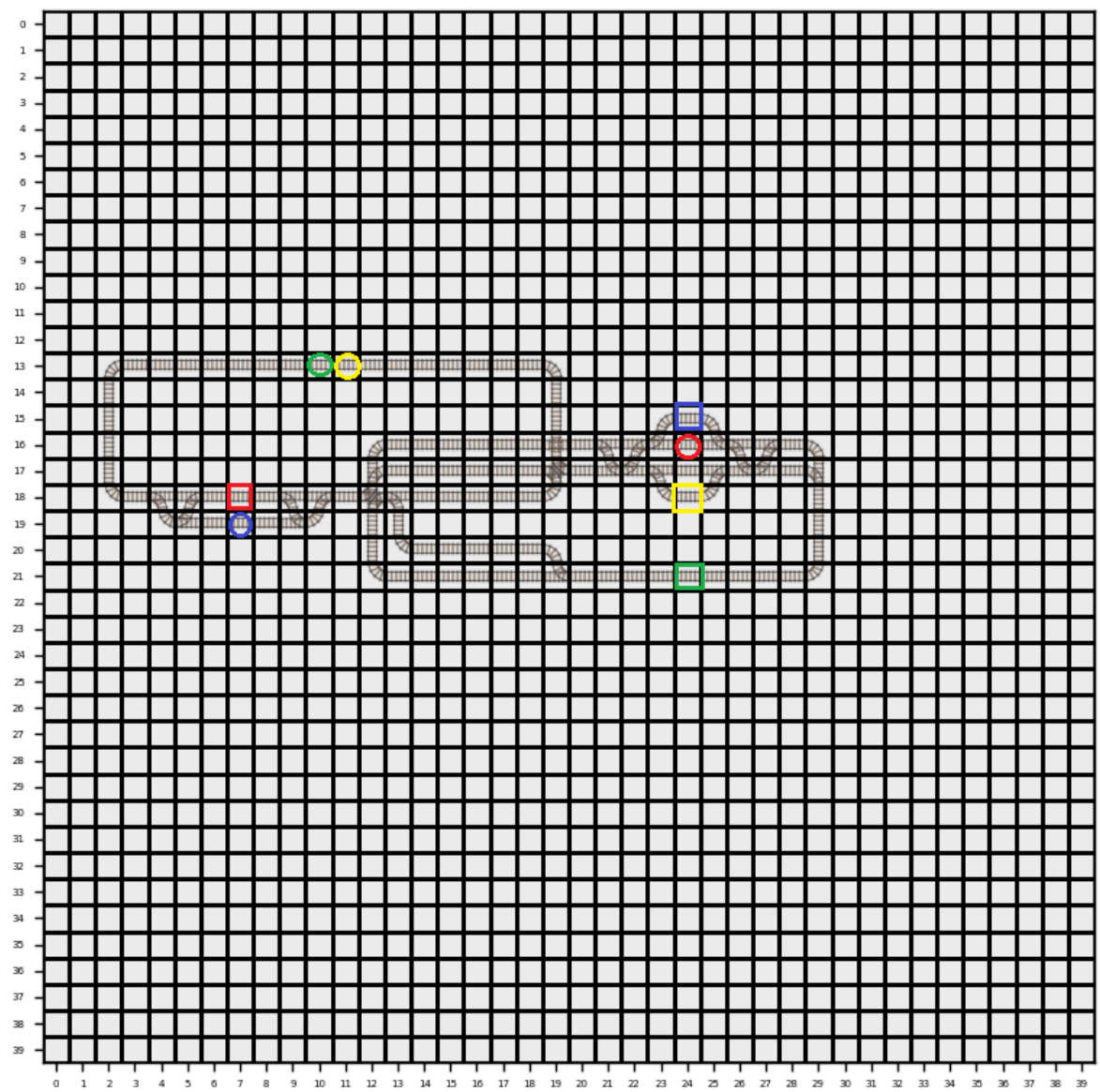


Trains:

```
train(0). start(0,(22,18),1,w). end(0,(24,32),49).  
train(1). start(1,(23,32),3,w). end(1,(23,18),28).  
train(2). start(2,(23,18),0,w). end(2,(25,26),28).  
train(3). start(3,(18,30),0,w). end(3,(18,32),38).
```

Complexity	Result
3 trains	Solves quickly. Finds 3 stable models.
4 trains	Unsatisfiable. Checking why: running only the single added train. It fails. So, the problem is in it alone. Not enough time was given. Running all 4 again: 4 stable models found

Kep’s Test 4:



Trains:

```
train(0). start(0,(18,7),3,w). end(0,(16,24),59).
train(1). start(1,(15,24),7,w). end(1,(19,7),42).
train(2). start(2,(21,24),0,w). end(2,(13,10),45).
train(3). start(3,(18,24),0,w). end(3,(13,11),60).
```

Complexity	Result
4 trains	Finds 2 stable models rather quickly and doesn't freeze.

Brainstorming Possible Extensions

1. Prioritization of trains.

Jonas has already implemented it in a very simple and effective way - a train with a higher ID is considered to have a higher priority. It works and works very fast - there's no need to reinvent it. But there might be opportunities to extend it based on the domain area's real challenges. Let's explore them through: a. Reading domain area sources; b. Consulting with an LLM.

Skimming:

<https://d-nb.info/1093877235/34>

Can't easily understand what they solve. Might be too low-resolution for our modelling level.

Consulting with an LLM (**GPT**):

1. Dynamic Priority Based on Context

Right now, priority is static (ID-based). In real systems, priority is **situational**:

- **Schedule adherence:** Trains closer to delay thresholds should get priority.
- **Type of train:** Cargo may yield to passenger trains.
- **Distance to destination:** Shortest remaining path might have higher priority.
- **Traffic load ahead:** A train heading into a congested area might be deprioritized.

➡ Extension: Implement **context-sensitive prioritization**, possibly with soft constraints or heuristics.

Mistral:

1. Train Type and Service Priority:

- **Passenger vs. Freight Trains:** Passenger trains often have higher priority due to their impact on human travel times and commitments.
- **High-Speed vs. Regular Trains:** High-speed trains may require prioritization to maintain their schedules and efficiency.

Connecting Services: Prioritizing trains that have many connecting services to minimize passenger wait times.

Deepseek:

Static priorities (e.g., higher ID = higher priority) may not handle real-world scenarios where:

- Some trains are **delayed** and need urgency boosts.
- **High-value cargo** (e.g., hazardous materials, express freight) should get precedence.
- **Passenger trains** might need priority over freight trains.

Possible Extensions:

- Add **dynamic priority weights** based on:
 - Train type (passenger > freight).
 - Remaining time to deadline (urgency-based).
 - Cargo importance (e.g., medical supplies).
- Implement **priority inheritance** (a delayed train temporarily inherits higher priority).

2. Deadlock Prevention & Resolution

Problem:

Even with prioritization, **circular waits** (deadlocks) can occur:

- Train A waits for Train B, which waits for Train C, which waits for Train A.
- Classic in grid-like networks with bidirectional tracks.

IDEA: let high-speed trains travel with as few stops as possible (isn't it a simple biggest ID?)

Stopping/starting trains wastes energy. Can we **minimize energy use** while preserving priority?

- Example: A lower-priority train might **slow down early** to avoid stopping.

This is a very interesting direction that works only together with implementing speed / acceleration.

Draft: I need to implement speed first. Then also implement acceleration.

2. Intermediate Stops:

Intuition:

Input looks like:

```
train(ID, ((Y,X), (Y,X), (Y,X), (Y,X), ...)) .
```

Then gets unpacked into 'many' trains following one another.

3.

Understanding Speed

It's still a question whether we all need to implement speed in our encodings. But it's worth understanding it / prototyping the basic implementation in case it's needed.

Speed is said to be **fractional**, which means:

22.06.25

... What does it mean? It's kind of the opposite of the 'normal' idea of speed. What is the normal idea of speed? 'How many units of distances are covered per unit of time'. Let's reverse it: 'how many units of time must pass to cover 1 unit of distance?' (not sure about it). Let's look up the moodle: it was probably in the pdf describing the project.

speed how long it takes an agent to execute an action

- in principle, different trains can have different speeds
- the slowest possible speed is 0
- the fastest possible speed is 1
- speeds are fractional, so a speed of
 - $\frac{1}{2}$ = one cell in two time steps
 - $\frac{1}{3}$ = one cell in three time steps
- Flatland ignores acceleration - a train always has the same speed when moving

But are there fractions in ASP? Let's see the input to the 4th test environment - speed was indicated there.

```
speed(0,1).
```

```
speed(1,1).
```

Yes, it is indicated, but it's just 1 (default) for both trains. Let's make sure fractions aren't possible in ASP:

Yes, they aren't explicitly. So here's what we have then:

`speed: speed(ID, InvSpd,`

As far as I understand, the second attribute is simply the denominator of the fraction representing speed, e.g.:

3 - means the speed is $\frac{1}{3}$, which means that the train covers 3 units of space in 1 unit of time.

2 - means the speed is $\frac{1}{2}$, which means that the train covers 2 units of space in 1 unit of time.

1 - means the speed is $\frac{1}{1}$, which means that the train covers 1 unit of space in 1 unit of time.

So, at least this is the clearest explanation of the 'inverse fractional speed' idea I can come up with.

Intuition for acceleration: it's still not continuous (a continuous value would lead to infinite grounding) - it's discretized into arbitrary steps:

$\frac{1}{9}$, $\frac{1}{7}$, $\frac{1}{6}$ etc.

Intuition: acceleration can be negative or positive or 0 (speed is constant). Asp allows for negative numbers.

Let's first focus on speed alone. For this, let's take my best encoding and the simplest single train environment.

Running it: solves it immediately. Now let's think where to plug in speed. First, let's just plug it into the input, without any actions taken to process the new facts:

```
train(1).  
speed(1,1).  
start(1, (2, 0), 0, north).  
end(1, (0, 0), 2).
```

Now the input looks like this. The new 'speed' atom reads: the train with ID 1 has the speed $\frac{1}{1}$, which means that the train covers 1 unit of space in 1 unit of time.

Now, let's try to make these atoms work. At which point in the encoding should I plug them in? 'Path' creation? Definitely not: 'path' just describes the environment tightly, without any movement happening in it. Then where?

Then probably in the generator:

```
1 { transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, TO_POS, OUT_DIR) :
path(IN_DIR, FROM_POS, ACTION, OUT_DIR, TO_POS) } 1 :- train(TRAIN),
moment(TRAIN, MOMENT) .
```

Where exactly? Now, let's just play around with speed being 1, so nothing really changes in the output. We probably should add 'speed' atom to the body of the rule:

```
1 { transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, TO_POS, OUT_DIR) :
path(IN_DIR, FROM_POS, ACTION, OUT_DIR, TO_POS) } 1 :- train(TRAIN),
moment(TRAIN, MOMENT), speed(TRAIN, MOMENTS_PER_POS) .
```

And then I need to make this atom affect the transitions created. What does 'affect' mean? It's not easy: my encoding is built in a way that a train is forced to make a transition, even if this transition is 'false' - like for 'wait' action. Let's model the possible outputs for different speed to know *what* we want to be generated and not to guess the choice rule blindly:

Speed	Output (straight line)
speed(TRAIN, 1)	transition(TRAIN,0,6,north,move_forward,3,north) transition(TRAIN,1,3,north,move_forward,0,north)
speed(TRAIN, 2)	transition(TRAIN,0,6,north,move_forward,3,north) transition(TRAIN,1,6,north,move_forward,3,north) transition(TRAIN,2,3,north,move_forward,0,north) transition(TRAIN,3,3,north,move_forward,0,north)
speed(TRAIN, 3)	transition(TRAIN,0,6,north,move_forward,3,north) transition(TRAIN,1,6,north,move_forward,3,north) transition(TRAIN,2,3,north,move_forward,0,north) transition(TRAIN,3,3,north,move_forward,0,north) transition(TRAIN,3,3,north,move_forward,0,north) transition(TRAIN,3,3,north,move_forward,0,north)

Looks not very elegant, but:

- I can view it as a train 'dwelling' on the same transition for a certain number of moments defined by its speed.
- The output would anyway look like a series of 'action' atoms that only contain a moment of time and what the train is doing - which would be a natural representation of speed.

Now, let's alter the choice rule in a way that makes such speed-bound generation possible.

Trying to modify the choice rule. The new logic is: for each moment of time defined by the ‘speed’ atom, generate a transition; OR: **“keep repeating the ‘transition’ with exactly the same attributes until the number of moments of time stated in ‘speed’ expires”**.

```
1 { transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, TO_POS, OUT_DIR) :
path(IN_DIR, FROM_POS, ACTION, OUT_DIR, TO_POS) } 1 :- train(TRAIN),
moment(TRAIN, MOMENT), speed(TRAIN, MOMENTS_PER_POS) .
```

I think I got the idea: ‘transition’ generation must be made in 2 steps:

1. Generate ‘actual’ transitions.

To do this, I need to think about the relationship between fractional speed and moments in the choice rule.

2. Fill out the gaps between them with copies.

‘Mod’ didn’t work in my solver. So, I imitated it with basic arithmetic:

```
moment(TRAIN, MOMENT) :- MOMENT = 0..MAX_TIME-1,
end(TRAIN, (_, _), MAX_TIME),
train(TRAIN),
speed(TRAIN, MOM_PER_POS),
R = MOMENT - (MOMENT / MOM_PER_POS) * MOM_PER_POS,
R == 0.
```

Let’s plug it into the big encoding and see the behaviour:

It works, but the output is weird. It might be weird because intermediate transitions aren’t created yet. Let’s try to implement them. What’s the rule for creating them? It’s normal rule; let’s formalize it:

Rule Idea	ASP phrasing	Code
If there are two transitions for the same train that are separated by a gap in moments of time, there must be created transitions with exactly the same attributes as the preceding transition but with moments of time connecting the two.	If there are two transition atoms for the same train between which there is no intermediate moment (they immediately follow each other), we should create an ‘intermediate_missing’ helping atom.	

Let's practise this 'filling out' logic on the simplest case possible outside of train movement logic:

```
atom(0, a). atom(2, b). atom(4, c). atom(6, d).
```

Subtask: create a normal rule that will fill out the missing 'atom' atoms with an identical attribute value, so that the output is (there must be atom 0 going first):

```
atom(0, a). atom(1, a). atom(2, b). atom(3, b). atom(4, c). atom(5, c).  
atom(6, d). atom(7, d).
```

Let's look at the input again:

```
atom(0, a). atom(2, b). atom(4, c). atom(6, d).
```

The logic must be roughly like this: if there is a missing step, plug in. Let's add a helping atom:

```
intermediate_missing(MOMENT).
```

How do we create this new added atom? With a normal rule:

If there are two atoms that follow each other (there is no atom in between them) - let's put it into an auxiliary atom:

```
immediately_follow(MOMENT_1, MOMENT_2) :- atom(MOMENT_1, _),  
atom(MOMENT_2, _), atom(MOMENT_3, _), MOMENT_1 < MOMENT_2, not MOMENT_1 <  
MOMENT_3 < MOMENT_2.
```

Good, I've learnt to do it here:

```
atom(0, a). atom(2, b). atom(4, c). atom(6, d).
```

```
all_time(7).
```

```
moment(0..MAX_TIME+1) :- all_time(MAX_TIME).
```

```
intermediate_missing(MOMENT) :- moment(MOMENT), not atom(MOMENT, _).
```

```
full_range_atom(MOMENT, TYPE) :- atom(MOMENT, TYPE).
```

```
full_range_atom(MOMENT, TYPE) :- atom(MOMENT-1, TYPE), intermediate_missing(MOMENT).
```

```
#show full_range_atom/2.
```

Now let's expand this logic onto transitions. it's hard: there is a conflict in asp that I don't understand.

I don't see where I'm missing. Let's try to play around with Jonas' encoding.

INTUITION: it may be easier to model speed through Jonas' 'at' logic. I already understand the concept fully - and there is a chance that it's just easier to implement in Jonas logic.

Let's just run it again first:

```
at(1,2,0,0,n) at(1,1,0,1,n) at(1,0,0,2,n) Works.
```

What I want to obtain is that the train dwells in every cell if its speed is lower than 1. Now, what atoms should I interfere in? The choice rule - the 'generator' of the program - produces 'action' atoms. So, I must interfere there. I've added the 'speed' fact.

It feels like it's much better to work together with Jonas on speed.

—

Let's step aside for a while and see what other problems (maybe, allowing us to avoid the difficulty of speed implementation):

Deadlocks:

What is a deadlock at all?

LLM:

A **deadlock on a railroad** occurs when two or more trains block each other in such a way that **none of them can move** without the others moving first, and **no train has a path forward**.

It's like a traffic jam where every vehicle blocks the one behind or beside it, but with trains, the situation is worse because they can't just steer around each other—they're stuck on fixed tracks.

Common causes include:

- Single-track lines without proper sidings for passing.
- Improper dispatching or signaling.
- Two trains entering a junction from opposing directions, neither able to proceed.

Example:

Imagine two trains on a single track coming from opposite directions. Both meet in the middle, and there's no siding nearby to let one pass the other. Neither can move without the other backing up, but that might not be possible due to other traffic or track layout.

Would you like a visual diagram, real-world example, or animation of how it happens?

But haven't we already solved the problem of deadlocks implicitly?